

FoundationDB and Its Use for A Graph Database

Hieu Nguyen
Software Engineer, eBay



Outline

- Part 1. FoundationDB
 - FoundationDB Deployment Architecture
 - FoundationDB Components
 - FoundationDB Transaction
 - Programming with FoundationDB
- Part 2. Graph Database
 - eBay Graph Database Architecture With FDB As the Backend
 - Graph Data Layout and Indices
 - Read and Write with Graph Database
- Part 3. Caching Techniques in Graph Database
 - Full Query Cache Template
 - One-Hop Query Cache Template
 - Implement Caching Techniques in Graph Database with FoundationDB
 - Multiple Stashes for the Backend

Part 1. FoundationDB

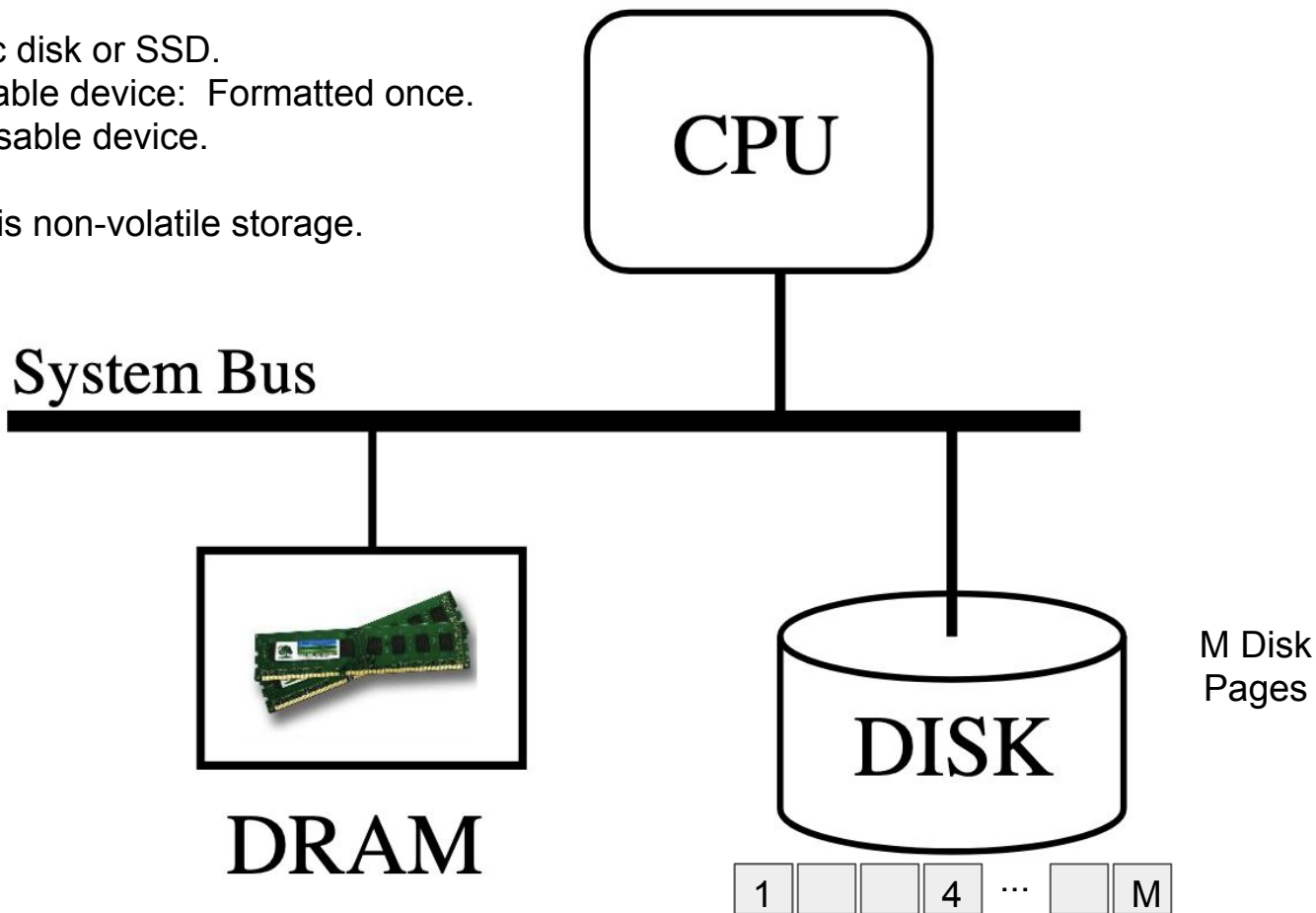
Hardware Organization

Disk may be a magnetic disk or SSD.

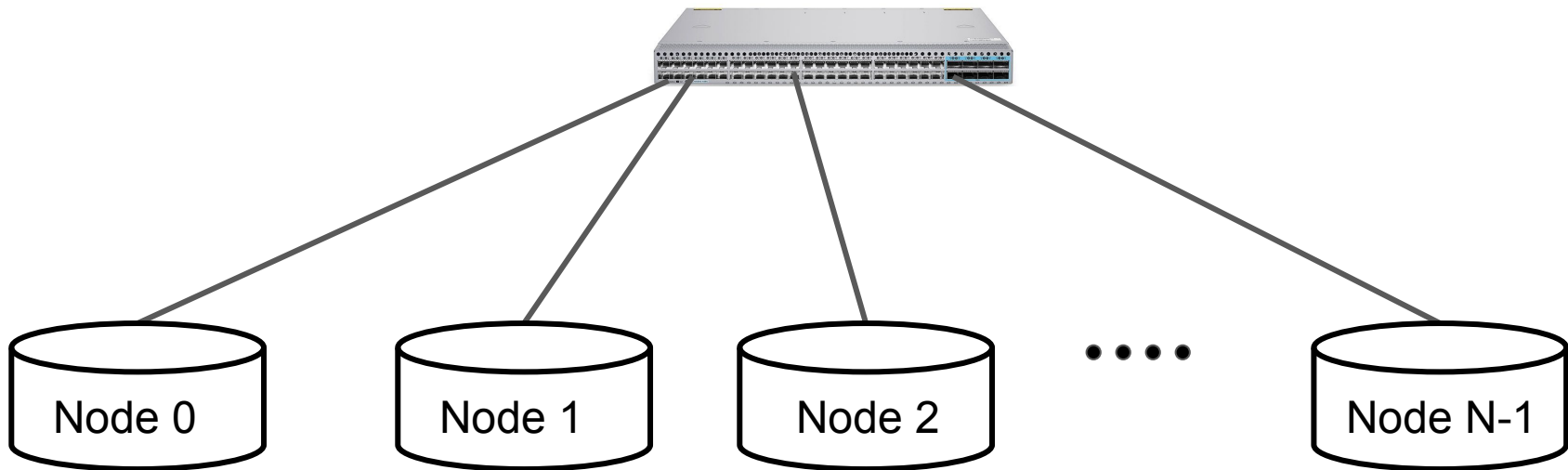
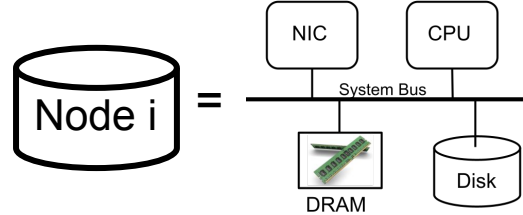
Disk is a block addressable device: Formatted once.

DRAM is a byte-addressable device.

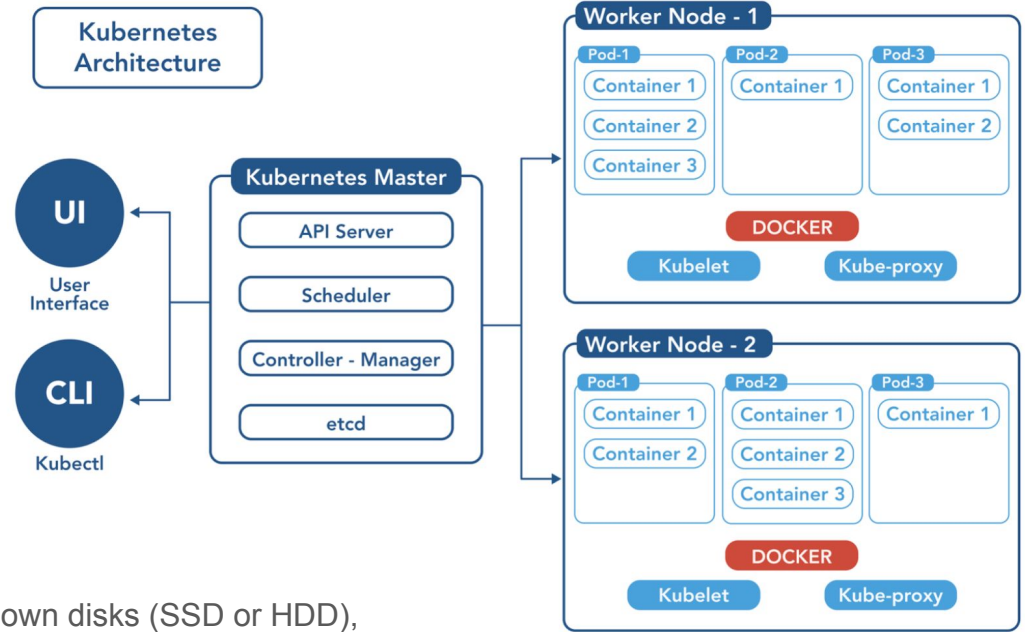
DRAM is volatile. Disk is non-volatile storage.



A Shared-Nothing Architecture

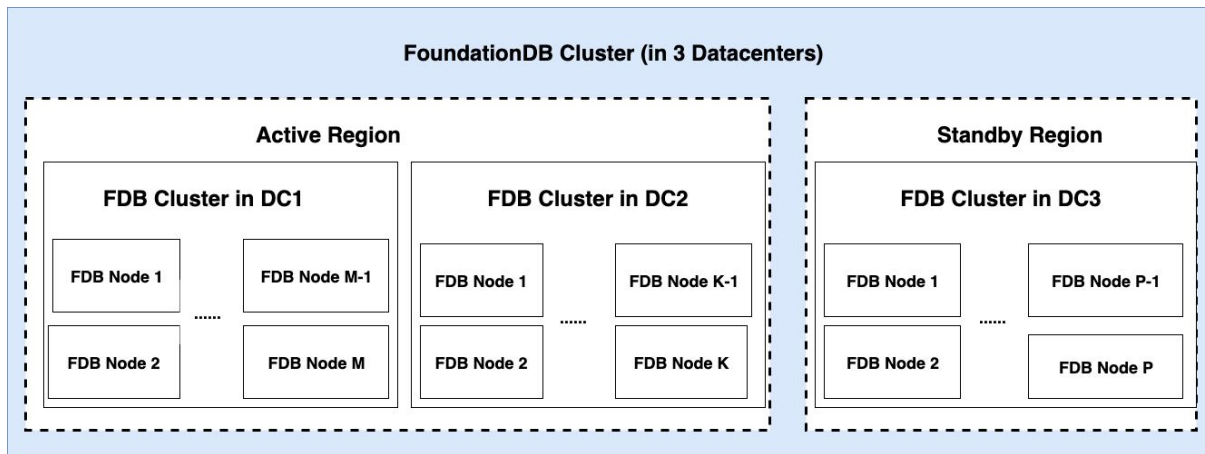


Kubernetes Architecture



- Multiple worker nodes
 - Physical (bare metal): Each node has its own disks (SSD or HDD), network, DRAM
 - Virtual machines (Cloud VMs)
- Each node may host multiple pods
- Each pod may have multiple containers

FoundationDB Deployment



- Hundreds of nodes per datacenter
- 3-datacenter, 2-region
 - Cross-DC latency: ~20-30ms
 - Inner-DC latency: 0.2ms
- Tolerate 1 DC Failure
 - DC1 fail - DC2 and DC3 form an active region
- A key-value pair is triple-replicated in each DC
 - 6 copies in total
 - Each DC can tolerate 2 storage node failures

FoundationDB - Distributed key-value store

- Keys are ordered
- Provide strict serializability
- MVCC: Multi-version concurrency control
- OCC: Optimistic concurrency control
- Scalable to hundreds of nodes
- Support multiple data-centers
- Layers can be built on top to support other data models (document, relational, graph)
- Acquired by Apple in 2015, open-sourced in 2018 under Apache 2.0 license



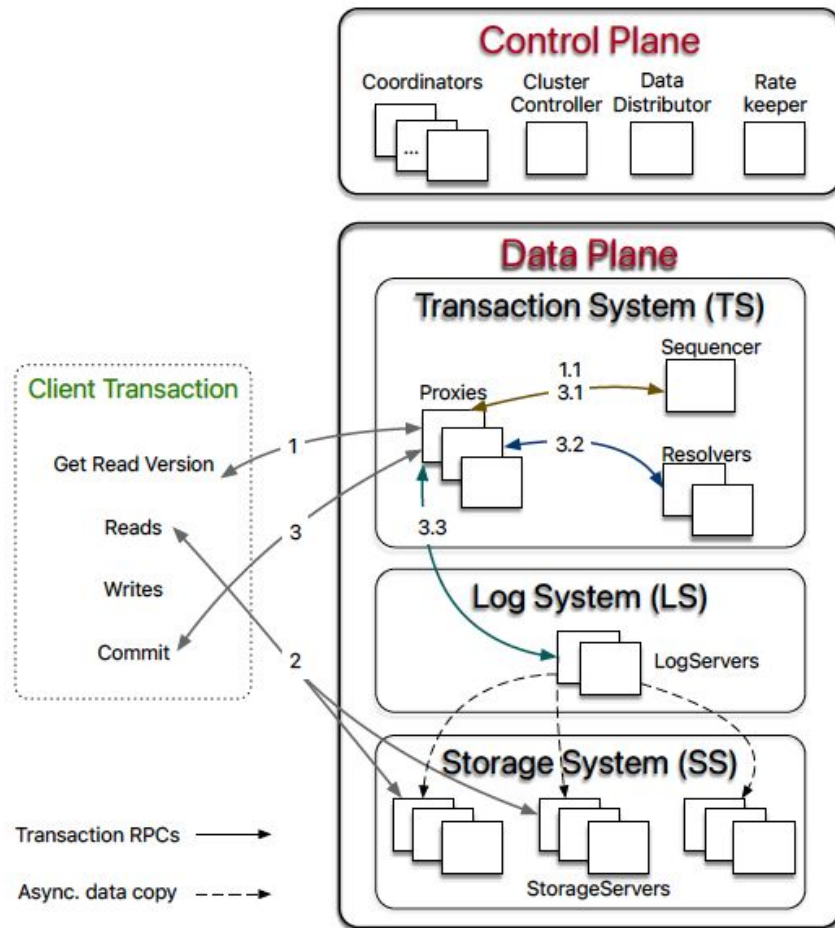
FoundationDB Components - "Divide-and-conquer"

Control Plane

- **Coordinators:**
 - Form a Paxos group
 - Recruit ClusterController, DataDistributor, RateKeeper
- **Singletons:**
 - **ClusterController:** Monitor all servers
 - **DataDistributor:** Monitor failures and balancing data among StorageServers
 - **RateKeeper:** Overload protection

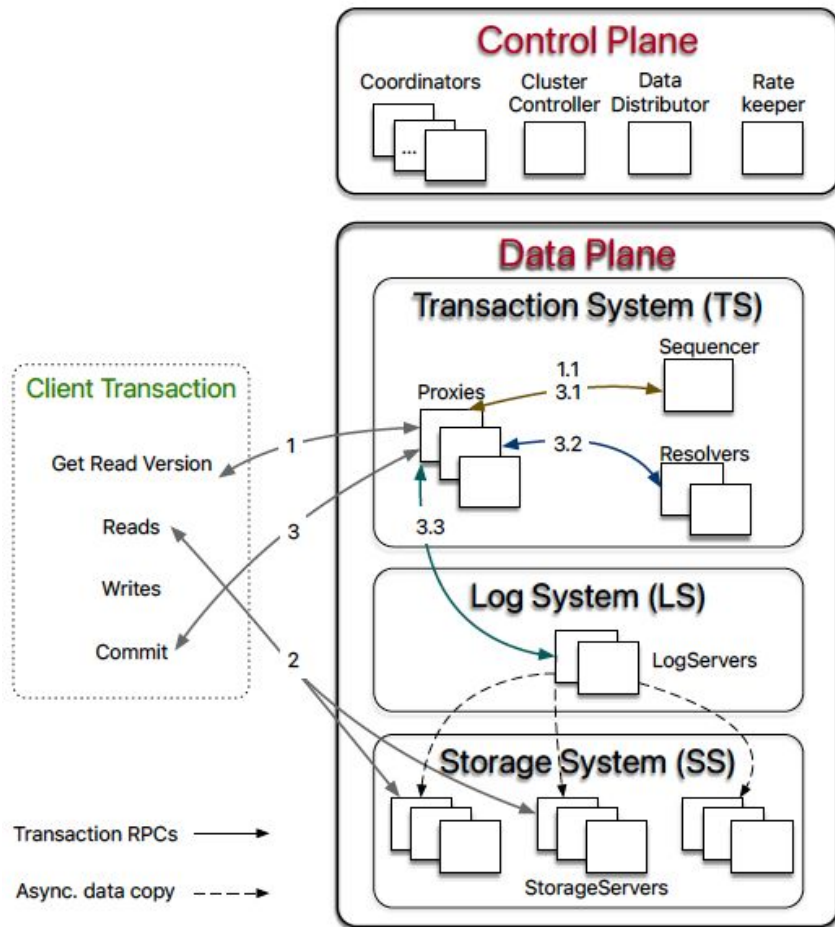
Data Plane

- **Transaction System (TS)**
 - **Sequencer** (singleton): Assign read and commit versions to transactions
 - **Proxies** (stateless): Offer MVCC read versions to clients and orchestrate transaction commits
 - **Resolvers** (range-based, stateless): Check for conflicts between transactions
- **Log System (LS):** LogServers
- **Storage System (SS):** StorageServers



Reconfiguration

2.3.5 Reconfiguration Whenever there is a failure in the TS or LS, or a database configuration change, a reconfiguration process brings the transaction management system to a new configuration, i.e., a clean state. Specifically, the Sequencer process monitors the health of Proxies, Resolvers, and LogServers. If any one of the monitored processes fails or the database configuration changes, the Sequencer process terminates. The ClusterController will detect the Sequencer failure event, then recruit a new Sequencer, which follows the above bootstrapping process to spawn the new TS and LS instance. In this way, transaction processing is divided into epochs, where each epoch represents a generation of the transaction management system with its unique Sequencer process.



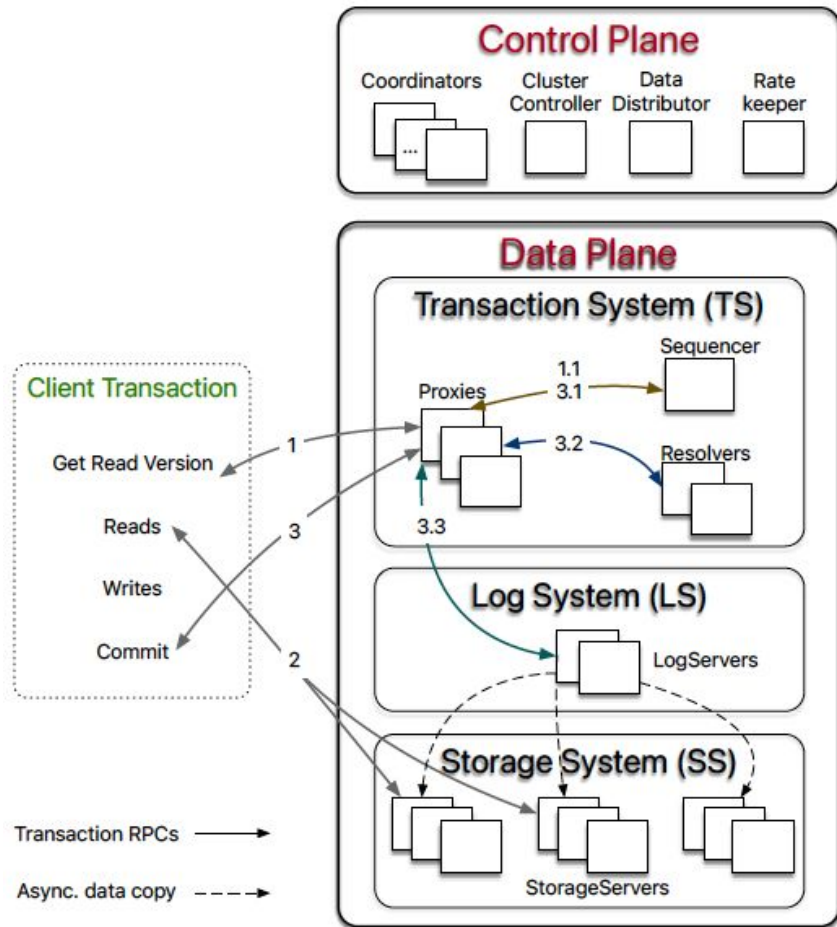
FDB Transaction

GetReadVersion

- (1) The client sends a request to a ProxyServer asking for a read version
- (1.1) The ProxyServer get the read version from Sequencer
 - It must be greater than the last committed version

Reads

- (2) The client issues reads directly to StorageServers
 - Cache the mapping between key ranges and storage servers



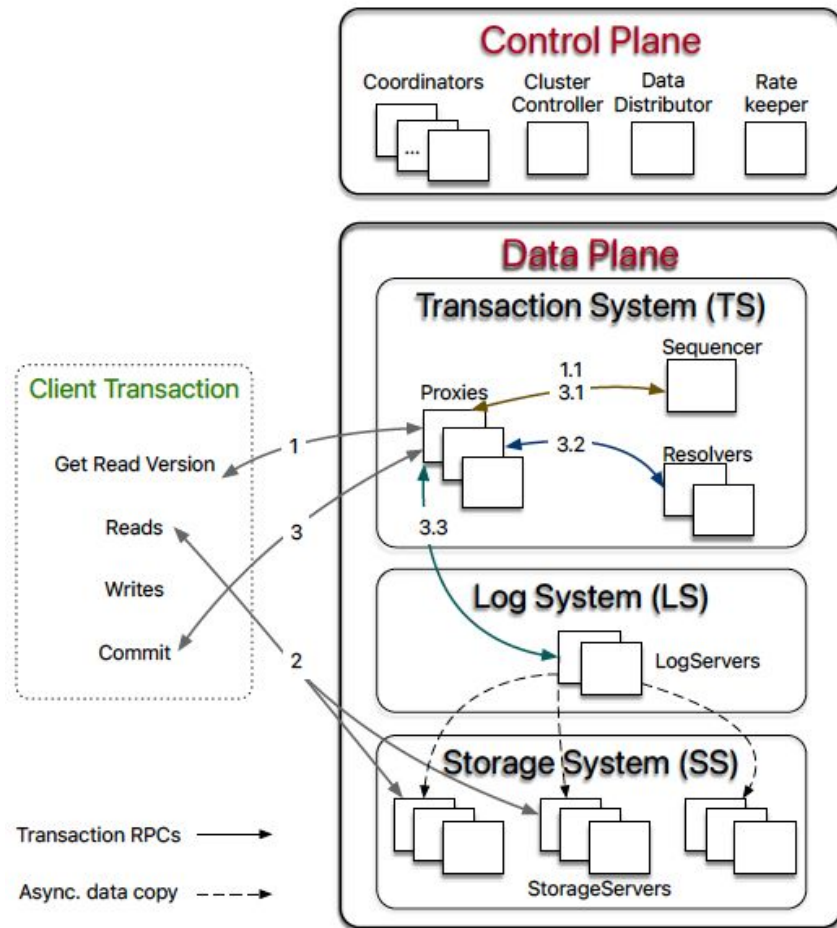
FDB Transaction (cont.)

Writes

- Buffer locally at the client

Commit

- (3) The client sends read and write sets to a ProxyServer
- (3.1) The Proxy asks the Sequencer for a commit version
- (3.2) The Proxy asks Resolver to check for read-write conflicts
 - Transaction can abort if a conflict is detected
- (3.3) The Proxy sends committed mutations to log servers for persistence.
 - A transaction is considered committed if designated LogServers replied to the Proxy
- (3.4) The Proxy reports the committed version to the Sequencer before return success to the client.



FDB Resolver - Conflict Checking

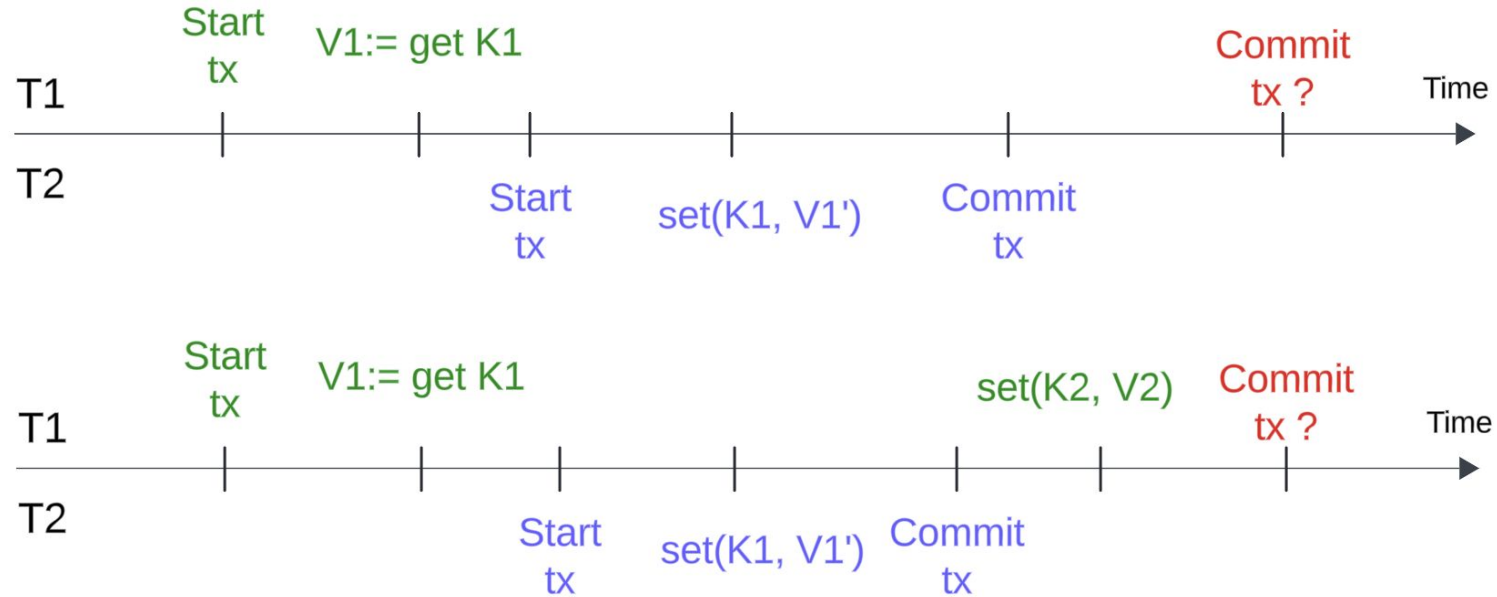
Algorithm 1 illustrates the lock-free conflict detection algorithm on Resolvers. Specifically, each Resolver maintains a history *lastCommit* of recently modified key ranges by committed transactions, and their corresponding commit versions. The commit request for T_x comprises two sets: a set of modified key ranges R_w , and a set of read key ranges R_r , where a single key is converted to a single key range. The read set is checked against the modified key ranges of concurrent committed transactions (line 1–5), which prevents phantom reads. If there are no read-write conflicts, Resolvers admit the transaction for commit and update the list of modified key ranges with the write set (line 6–7). For snapshot reads, they are not included in the set R_r . In practice, *lastCommit* is represented as a version-augmented probabilistic SkipList [56].

Algorithm 1: Check conflicts for transaction T_x .

Require: *lastCommit*: a map of key range $\rightarrow$ last commit version

```
1 for each  $range \in R_r$  do
2    $ranges = lastCommit.intersect(range)$ 
3   for each  $r \in ranges$  do
4     if  $lastCommit[r] > T_x.readVersion$  then
5       return abort;
// commit path
6 for each  $range \in R_w$  do
7    $lastCommit[range] = T_x.commitVersion;$ 
8 return commit;
```

T1 Commits or Aborts?



FDB Logging Protocol

2.4.3 Logging Protocol After a Proxy decides to commit a transaction, the log message is broadcast to all LogServers. As illustrated in Figure 2, the Proxy first consults its in-memory shard map to determine the StorageServers responsible for the modified key range. Then the Proxy attaches StorageServer tags 1, 4, and 6 to the mutation, where each tag has a preferred LogServer for storage. In this example, tags 1 and 6 have the same preferred LogServer. Note the mutation is only sent to the preferred LogServers (1 and 4) and an additional LogServer 3 to meet the replication requirements. All other LogServers receive an empty message body. The log message header includes both LSN and the previous LSN obtained from the Sequencer, as well as the known committed version (KCV) of this Proxy. LogServers reply to the Proxy once the log data is made durable, and the Proxy updates its KCV to the LSN if all replica LogServers have replied and this LSN is larger than the current KCV.

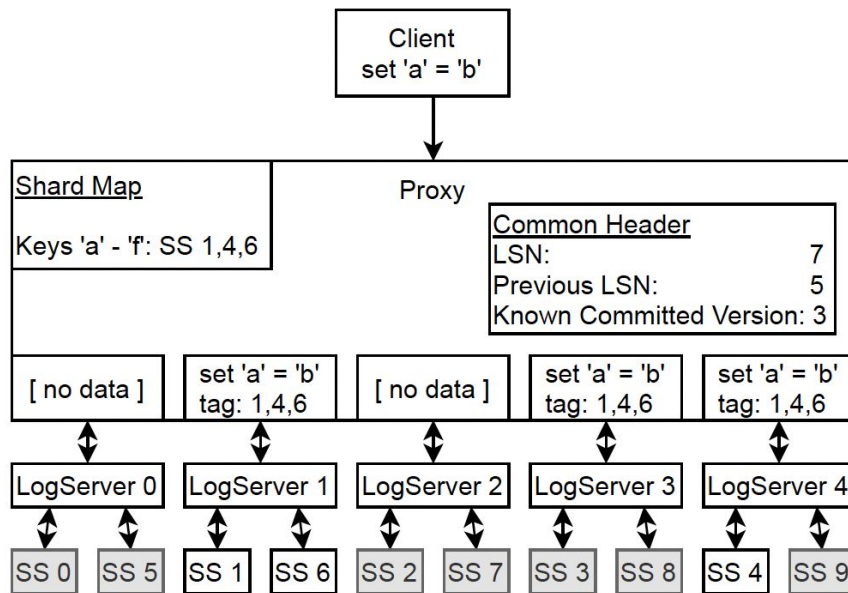


Figure 2: Proxy writes a client mutation to LogServers after sequencing and resolution. Later, the mutation will be asynchronously replicated to StorageServers.

FDB Storage Server

- Storage server maintains versions of data in the last 5 seconds in-memory
- The latest applied value of each key is stored persistently on KV engine (SQLite)

How FDB Storage Server Serves a Read Request?

LastAppliedVersion: 10

Key X:

1	3	6
---	---	---

Key Y:

2	7
---	---

Get(X, 4): The largest version ≤ 4 is chosen, i.e. version 3 of key X

Get(Y, 1): Error("Version too old"), as there is not any version ≤ 1

Get (X, 6): Version 6 is chosen

Get(Y, 8): Version 7 is chosen

Get(Y, 11): Error("Future version")

Storage Server Serving a Read Request

What if a StorageServer is lagging behind on applying the redo logs and a client requests a version of a key pair it does not have?

1. Wait for a threshold for when known-committed-version is greater than or equal to the read version
2. If timeout, the client asks another StorageServer that stores the key
3. Return error "request for a future version" (FDB error code 1009)

What if there is no further transaction logs to redo?

- Without new transactions issued from the client, proxies still generate empty transactions to advance the known-committed-version
- Known-committed-version and LSN of each transaction are sent to all LogServers (limit scalability on writes)

FoundationDB - Quizz

What does Commit do in a read-only transaction (no write)?

The client skips the commit protocol if there is no write.

Is it possible multiple resolvers resolve conflicts for a transaction?

Yes.

What are the consequences if one resolver detects a conflict while the others do not?

The transaction is aborted and other resolvers may potentially false-positive detect conflicts on future transactions

What if a StorageServer is lagging behind on applying the redo logs and a client requests a version of a key pair it does not have?

1. Wait for a threshold for the version to be there
2. If timeout, the client asks another StorageServer that stores the key
3. Return error "request for a future version" (FDB error code 1009)

FDB Programming Example

Emp

ssnum	name	age	salary	dno
123	Mary	22	55000	3
323	Joe	33	34000	1
324	John	40	55000	2
333	Mike	25	60000	1
555	Alice	45	80000	1
666	Bob	50	90000	2

Dept

dno	dname	floor
1	shoe	1
2	toy	1
3	men	2
4	women	2

FDB Programming

Get:

- `CompletableFuture<byte[]> get(byte[] key)`

Set:

- `void set(byte[] key, byte[] value)`

GetRange:

- `AsyncIterable<KeyValue> getRange(byte[] start, byte[] end)`
- `AsyncIterable<KeyValue> getRange(Range range)`

Clear:

- `void clear(byte[] key)`
- `void clear(byte[] start, byte[] end)`
- `void clear(Range range)`

FDB Programming Example (cont.)

```
public static Emp[] emps = new Emp[] { 1 usage
    new Emp() {{ssNum = "123"; name = "Mary"; age = 22; salary = 55000; dNo = 3;}},
    new Emp() {{ssNum = "323"; name = "Joe"; age = 33; salary = 34000; dNo = 1;}},
    new Emp() {{ssNum = "324"; name = "John"; age = 40; salary = 55000; dNo = 2;}},
    new Emp() {{ssNum = "333"; name = "Mike"; age = 25; salary = 60000; dNo = 1;}},
    new Emp() {{ssNum = "555"; name = "Alice"; age = 45; salary = 80000; dNo = 1;}},
    new Emp() {{ssNum = "666"; name = "Bob"; age = 50; salary = 90000; dNo = 2;}},
};
```

```
public static Dept[] depts = new Dept[] { 1 usage
    new Dept() {{dNo = 1; dName = "shoe"; floor = 1;}},
    new Dept() {{dNo = 2; dName = "toy"; floor = 1;}},
    new Dept() {{dNo = 3; dName = "men"; floor = 2;}},
    new Dept() {{dNo = 4; dName = "women"; floor = 2;}},
};
```

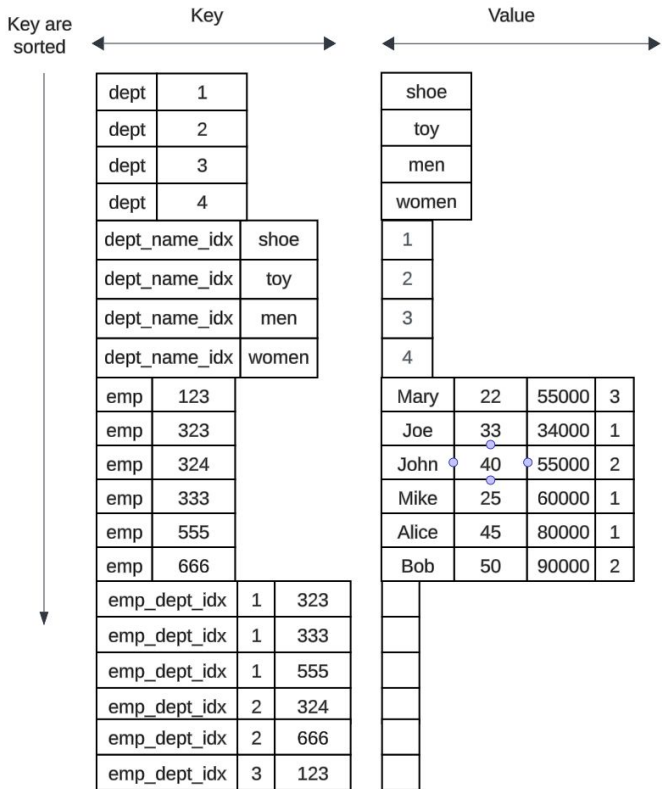
FDB Programming Example - Load Data

```
// Load emps and build index
```

```
db.run(tr -> {  
  for (Emp emp: emps) {  
    Tuple empKey = Tuple.from( ...items: "emp", emp.ssNum);  
    tr.set(empKey.pack(), Tuple.from(emp.name, emp.age, emp.salary, emp.dNo).pack());  
  
    // Build index for emp by dNo  
    Tuple empIdxKey = Tuple.from( ...items: "emp_dept_idx", emp.dNo, emp.ssNum);  
    tr.set(empIdxKey.pack(), Tuple.from().pack());  
  }  
  return null;  
});
```

```
// Load depts and build index
```

```
db.run(tr -> {  
  for (Dept dept: depts) {  
    Tuple deptKey = Tuple.from( ...items: "dept", dept.dNo);  
    tr.set(deptKey.pack(), Tuple.from(dept.dName, dept.floor).pack());  
  
    // Build index for dept by dName  
    Tuple deptIdxKey = Tuple.from( ...items: "dept_name_idx", dept.dName, dept.dNo);  
    tr.set(deptIdxKey.pack(), Tuple.from().pack());  
  }  
  return null;  
});
```

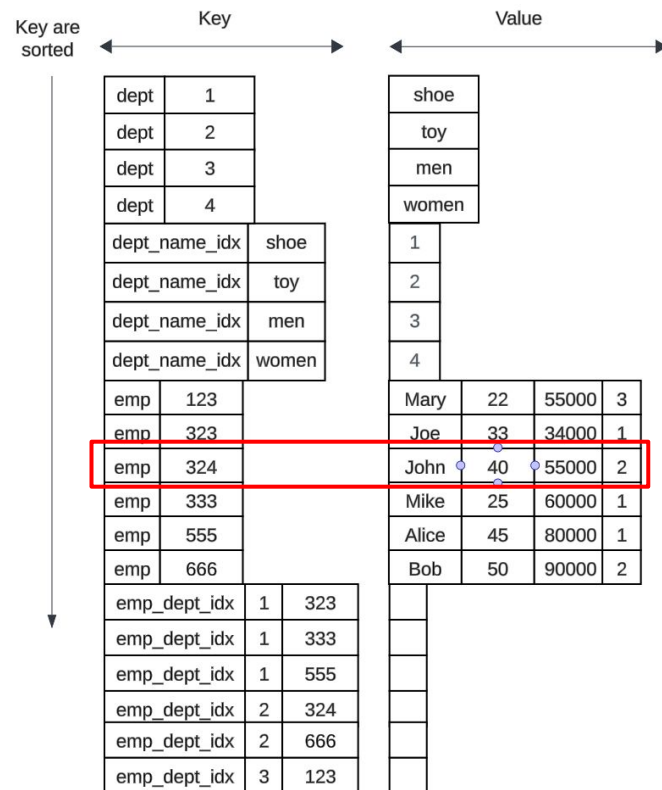


FDB - Queries

Get an employee given her ssNum

getEmp(db, 324)

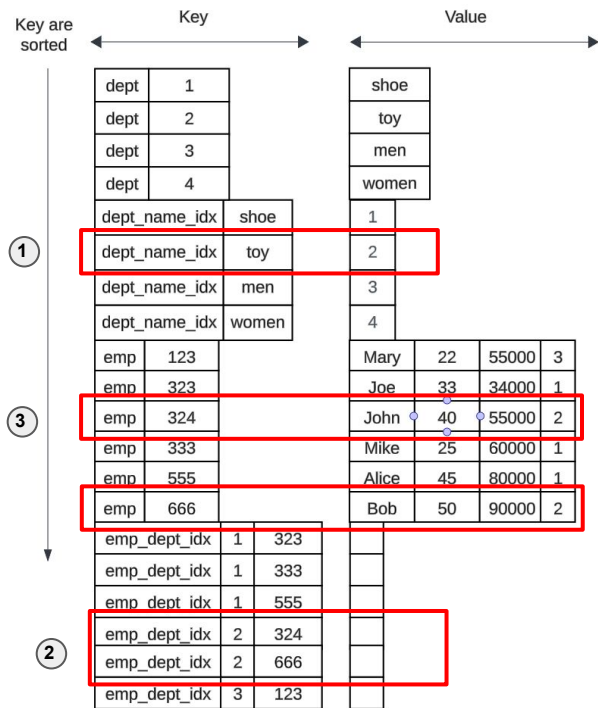
```
public static void getEmp(Database db, String ssNum) { no usages
    db.run(tr -> {
        Tuple empKey = Tuple.from( ...items: "emp", ssNum);
        byte[] empVal = tr.get(empKey.pack()).join();
        Tuple empValTuple = Tuple.fromBytes(empVal);
        String name = empValTuple.getString( index: 0);
        int age = empValTuple.getBigInteger( index: 1).intValue();
        long salary = empValTuple.getLong( index: 2);
        log.info("Emp: name={}, age={}, salary={}", name, age, salary);
        return null;
    });
}
```



FDB - Queries (cont.)

Get all employees in a department given the dept name

getEmpsByDeptName(db, "toy")



```
public static void getEmpsByDept(Transaction tr, int dNo) { 1 usage
    Tuple empIdxKey = Tuple.from( ...items: "emp_dept_idx", dNo);
    Range range = Range.startsWith(empIdxKey.pack());
    2 List<KeyValue> empIdxVals = tr.getRange(range).asList().join();
    for (KeyValue kv: empIdxVals) {
        // Extract the emp ssNum
        Tuple empIdxKeyTuple = Tuple.fromBytes(kv.getKey());
        String ssNum = empIdxKeyTuple.getString( index: 2);
        3 getEmp(tr, ssNum);
    }
}
```

```
public static void getEmpsByDeptName(Database db, String dName) { 1 usage
    db.run(tr -> {
        Tuple deptIdxKey = Tuple.from( ...items: "dept_name_idx", dName);
        Range range = Range.startsWith(deptIdxKey.pack());
        1 List<KeyValue> deptIdxVals = tr.getRange(range).asList().join();
        for (KeyValue kv: deptIdxVals) {
            // Extract the dept dNo
            Tuple deptIdxKeyTuple = Tuple.fromBytes(kv.getKey());
            int dNo = deptIdxKeyTuple.getBigInteger( index: 2).intValue();
            getEmpsByDept(tr, dNo);
        }
    });
}
```

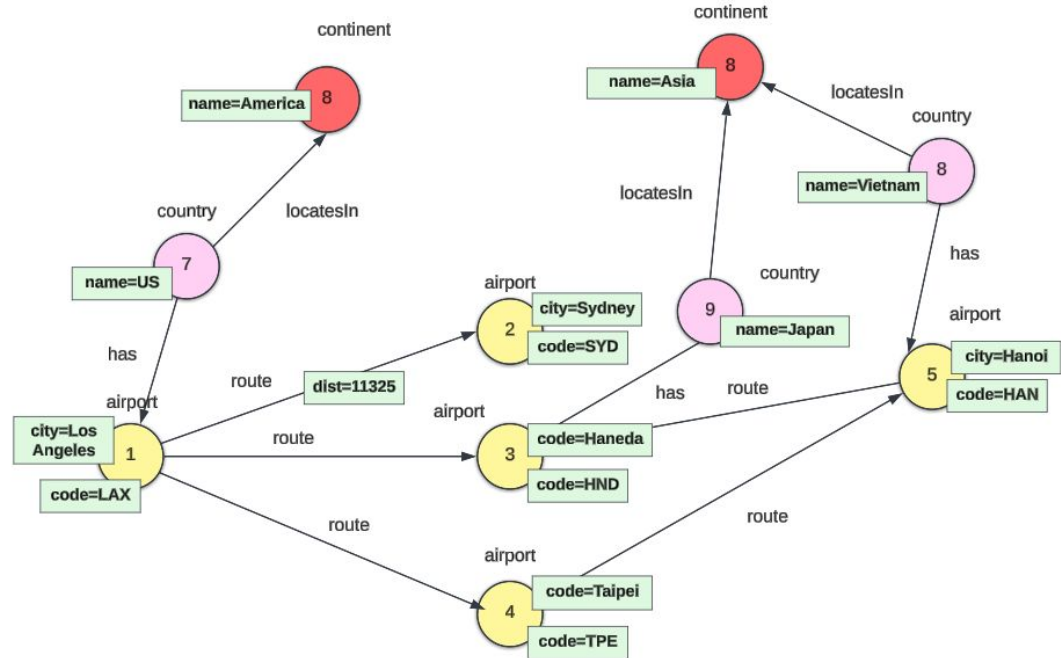
Data Model Built Upon FDB

- Relational:
 - FDB Record Layer: <https://arxiv.org/abs/1901.04452>
- Document:
 - <https://foundationdb.github.io/fdb-document-layer/>
- **Graph:**
 - **JanusGraph that uses FDB as its storage backend**

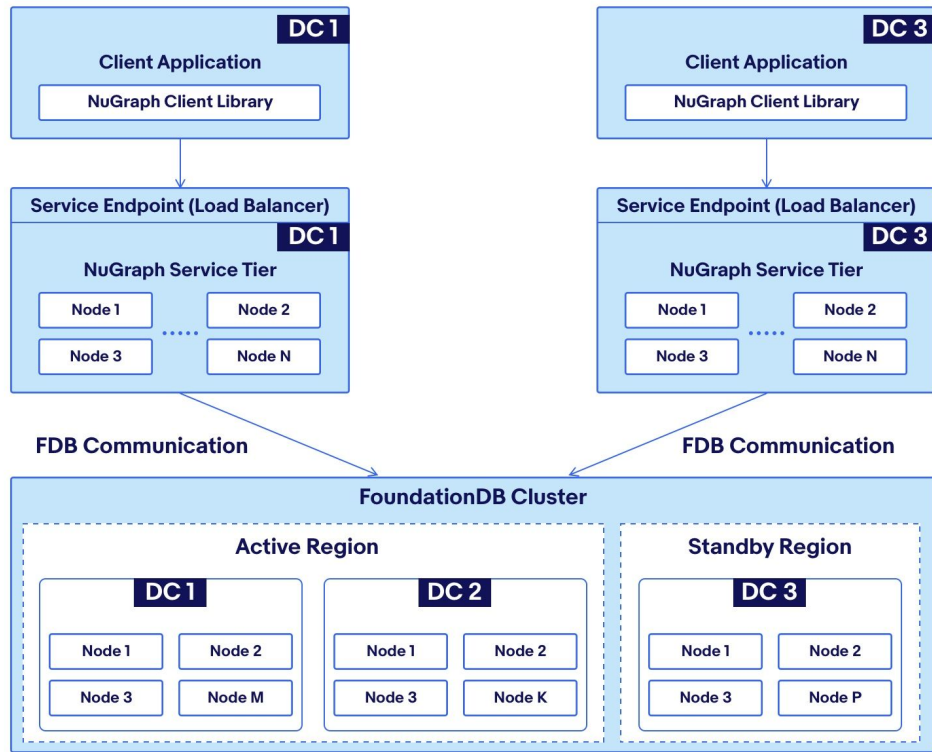
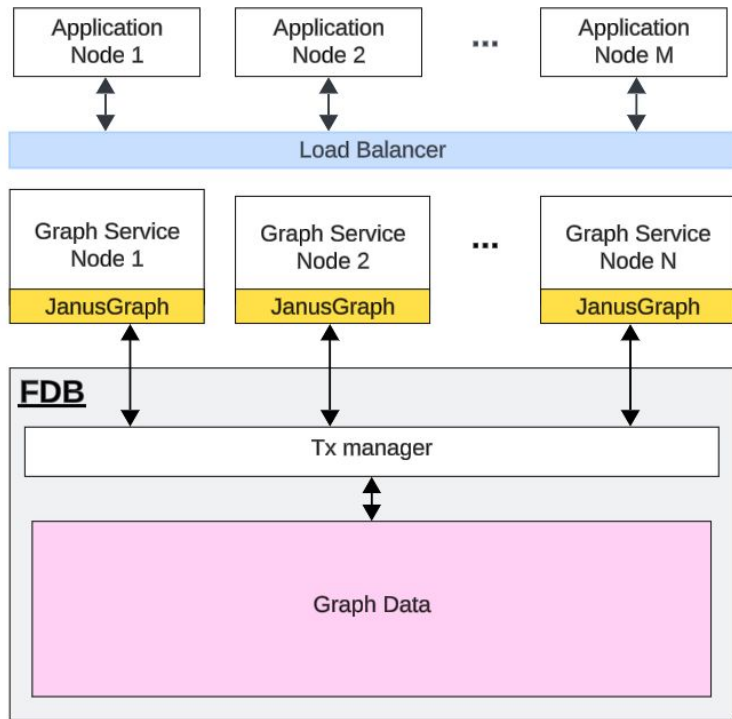
Part 2. Graph Database

A Graph Example

- Graph elements include vertices (nodes) and edges
- Each vertex/edge has one label and zero, one or more properties
- An edge connects two vertices



Graph Architecture



JanusGraph



- Open-sourced
- Graph database engine
 - Storage backend: FoundationDB, Cassandra, HBase, BerkeleyDB, etc..
 - Tinkerpop: A graph computing framework with Gremlin query language
- Horizontally Scalable
- Designed to manage billion of vertices and edges

Get familiar with Gremlin queries

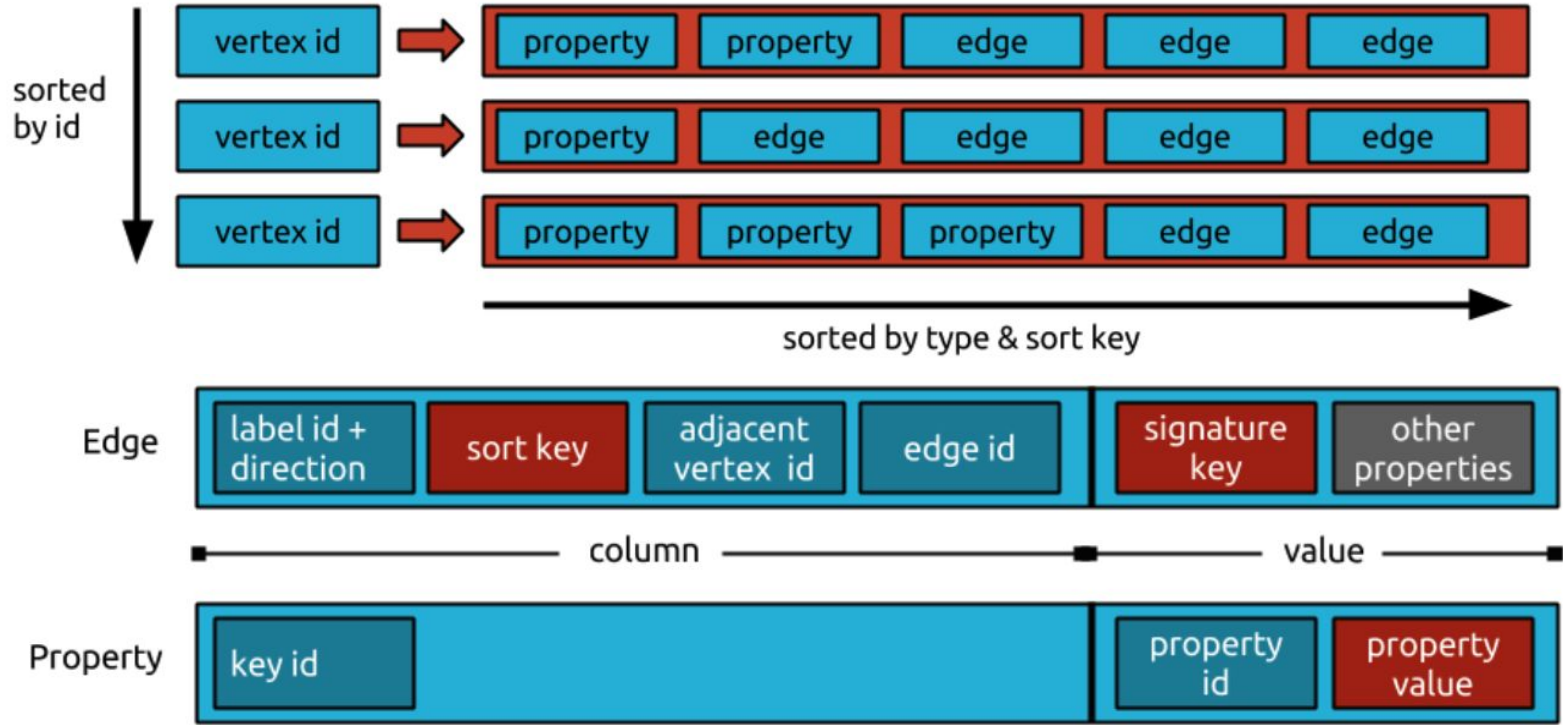
- Find an airport with code "LAX" and retrieve all its information
 - `g.V().has("airport", "code", "LAX").valueMap()`
- Find all airports that "LAX" can directly connect to
 - `g.V().has("airport", "code", "LAX").outE("route").inV().valueMap()`
- Find all airports that connect directly to "LAX"
 - `g.V().has("airport", "code", "LAX").inE("route").outV()`
- Find all possible 2-hop flights from Hanoi to Los Angeles

```
g.V().has('airport', 'city', 'Hanoi')
    .out('route').out('route').has('city', 'Los Angeles')
    .path()
        .by('code')
        .by(valueMap('city', 'code'))
        .by('code')
```

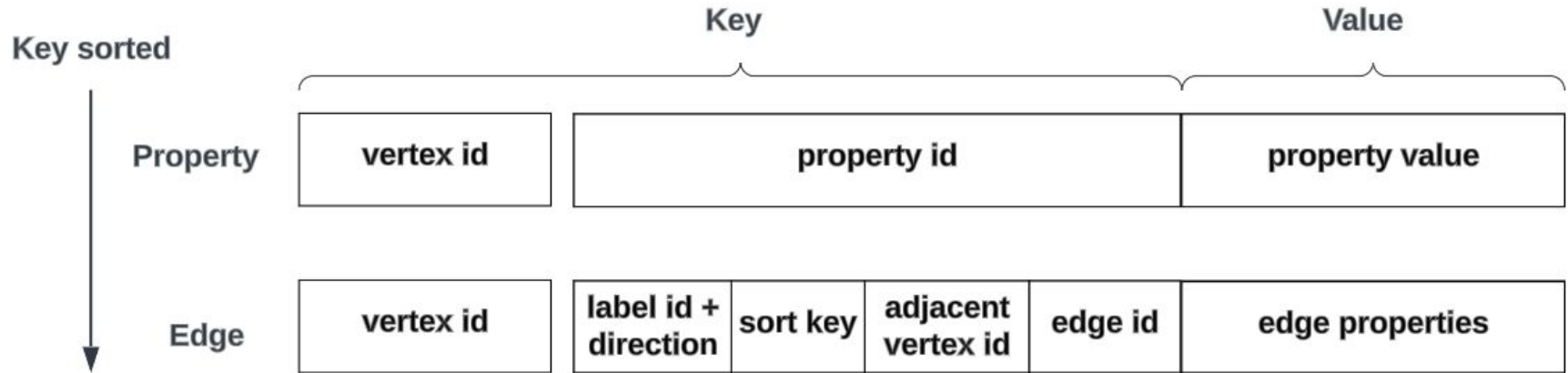
```
==>path[HAN, {code=[PVG], city=[Shanghai]}, LAX]
==>path[HAN, {code=[CDG], city=[Paris]}, LAX]
==>path[HAN, {code=[SYD], city=[Sydney]}, LAX]
```

```
...
```

JanusGraph Data Layout - Key-Column-Value



JanusGraph Data Layout - Key-Value



Graph Operation - Add a vertex

```
g.addV("airport")  
  .property("city", "Los Angeles")  
  .property("code", "LAX")  
g.addV("airport")  
  .property("city", "Sydney")  
  .property("code", "SYD")
```

Key		Value
1	city	Los Angeles
1	code	LAX
1	_label	90
3	city	Sydney
3	code	SYD
3	_label	90

Graph Operation - Add an edge

```
g.addE("route")
```

```
.property("dist", 11235")
```

```
.from(V(1)).to(V(3))
```

- "2" is the edge id in this example
- Add 2 keys that represents this edge

1	city					Los Angeles
1	code					LAX
1	label					90
1	route-out		3	2		dist=11235
3	city					Sydney
3	code					SYD
3	label					90
3	route-in		1	2		dist=11235

Graph Operation - Get all properties of a vertex

`g.V(1).valueMap()`

- 1 getRange

1	city				Los Angeles
1	code				LAX
1	_label				90
1	route-out		3	2	dist=11235
3	city				Sydney
3	code				SYD
3	_label				90
3	route-in		1	2	dist=11235

Graph Operation - Get all outgoing edges of a vertex

`g.V(1).outE("route").valueMap()`

- 1 getRange

1	city				Los Angeles
1	code				LAX
1	_label				90
1	route-out		3	2	dist=11235
3	city				Sydney
3	code				SYD
3	_label				90
3	route-in		1	2	dist=11235

Delete a vertex

g.V(1).drop()

- Get all edges of V1
 - Extract adjacent vertices (3)
- Remove all keys with prefix 1
 - clearRange(Range.startsWith(1))
- Remove key from 3 that represents the edge connected with 1
 - clear("3-route-in-1-2")

1	city					Los Angeles
1	code					LAX
1	_label					90
1	route-out		3	2		dist-11235
3	city					Sydney
3	code					SYD
3	_label					90
3	route-in		1	2		dist-11235

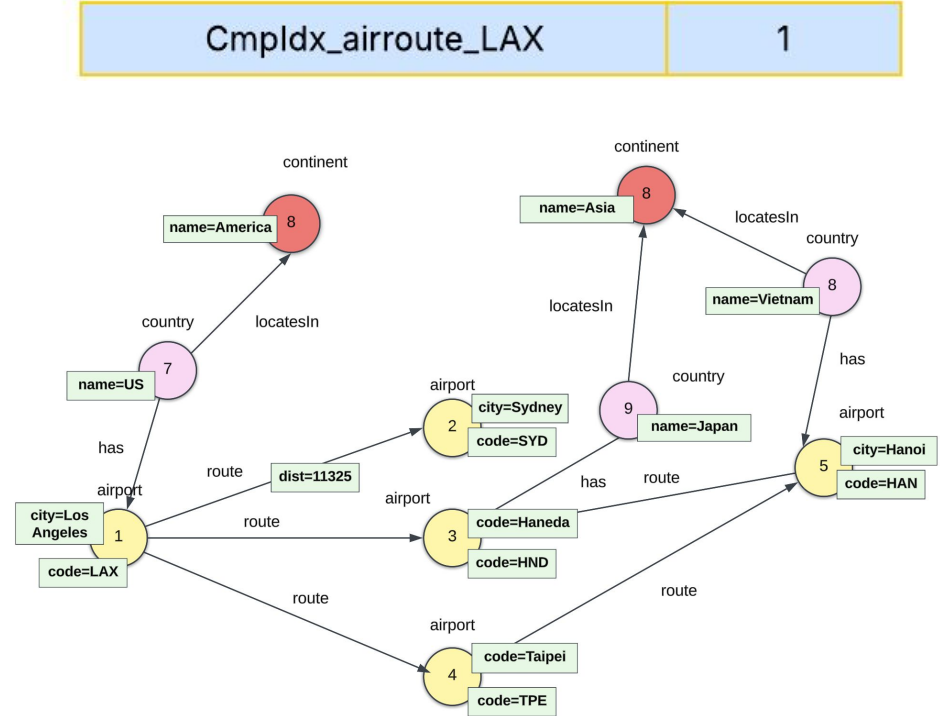
JanusGraph Indices

- Graph indices
 - Composite index
 - Hash-based
 - Mixed indices
 - External indices
 - To support global range queries and text search
 - ElasticSearch, Solr
 - **Can we incorporate them in FDB?**
- Vertex-centric Index
 - Local index (per vertex)
 - Deal with supernode issue where a vertex may have million of edges connect to it.

Composite Index Example

`g.V().hasLabel("airport").has("code", LAX")`

- Composite index on label="airport" and property "code"
- One lookup (get) to find the vertex id(s)

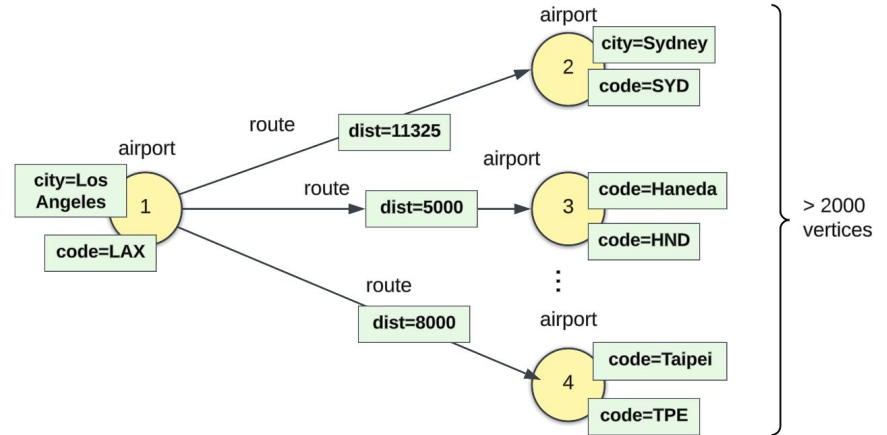


Vertex-Centric Index Example

```
g.V().hasLabel("airport").has("code",  
"LAX").outE("route").has("dist",  
P.lte(6000)).inV().values('code')
```

<u>Key</u>					<u>Value</u>
1	city				<value>
1	code				<value>
1	route-out	dist=11325	2	<edge_id>	<value>
1	route-out	dist=8000	4	<edge_id>	<value>
1	route-out	dist=5000	3	<edge_id>	<value>

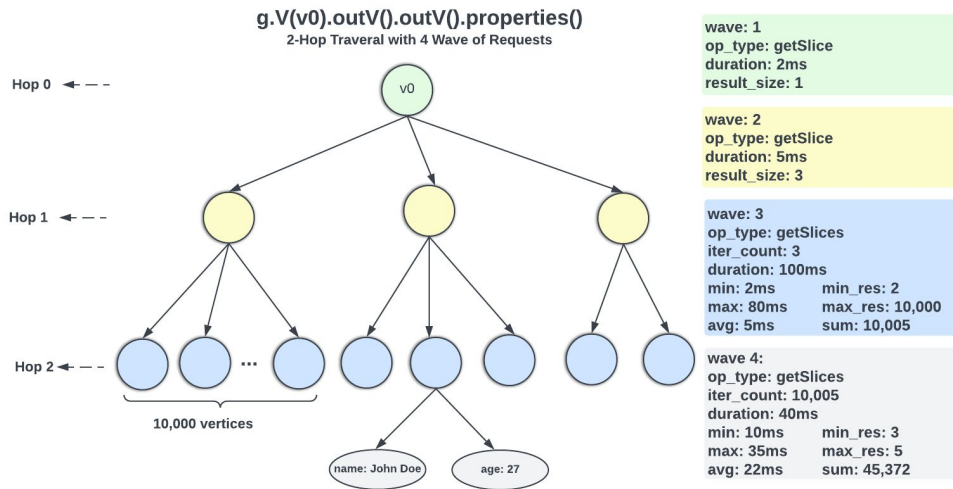
- Vertex-centric on edge label "route" and property "dist"
 - Edges are sorted by "dist" in FDB
- getRange for outE with a narrower range
- Deal with traversing supernode



Part 3. Caching Techniques in Graph Database

Motivation - Multi-Hop Query High Latency

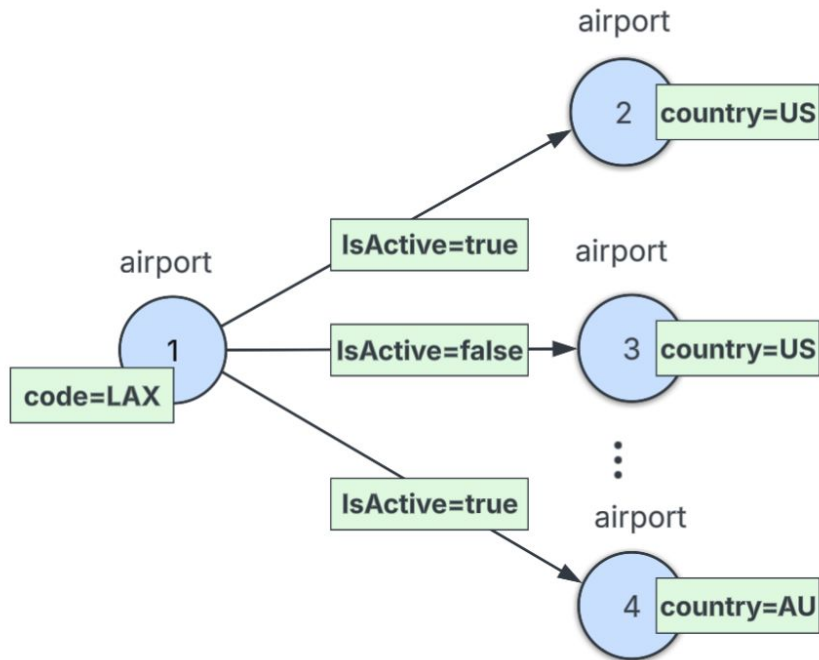
- A graph query could have multiple wave of requests
- In each wave, multiple `getRange()` is sent in parallel (in one or multiple batches) to the backend data store.
- The latency in a wave depends on the number of `getRange()` requests and the latency of each request.
- Typically, one big wave contributes significantly (mostly) to the graph query latency.
- The query profiling shows the total number of waves, and the stats of each wave of requests



High Latency Scenario 1

- LAX is a supernode
- A large number of its neighbors are examined
- Graph processing:
 - 1 **getRange** to look up the index for vertex id with code='LAX'
 - 1 **getRange** (big) to fetch all edges of LAX
 - Deserialize byte data to get the edges and filter the ones with IsActive=true
 - Collect the neighbor vertex ids
- Cache approach:
 - Query result (neighbor vertex ids) is stored in a cache entry
 - Key: <QueryTemplate>:1:IsActive=true
 - Next time the same query comes:
 - 1 **getRange** to look up the index for vertex id with code='LAX'
 - Look up the cache to collect the neighbor vertex ids

Find all airports with active flight routes from LAX
`g.V().has('code', 'LAX').outE().has('IsActive', 'true').inV()`



High Latency Scenario 2

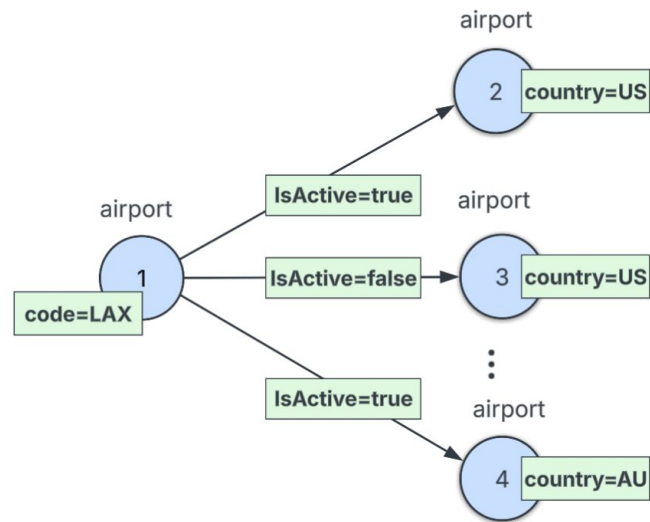
- For the filtered neighbors from Scenario 1, do extra filter on their properties for country='AU'
- Graph Processing:
 - Issue **one *getRange* for each neighbor vertex** (based on its id) to fetch the property and filter the one with country='AU'
 - 1 + 1 + (N extra getRange requests sent in parallel)

Cache approach:

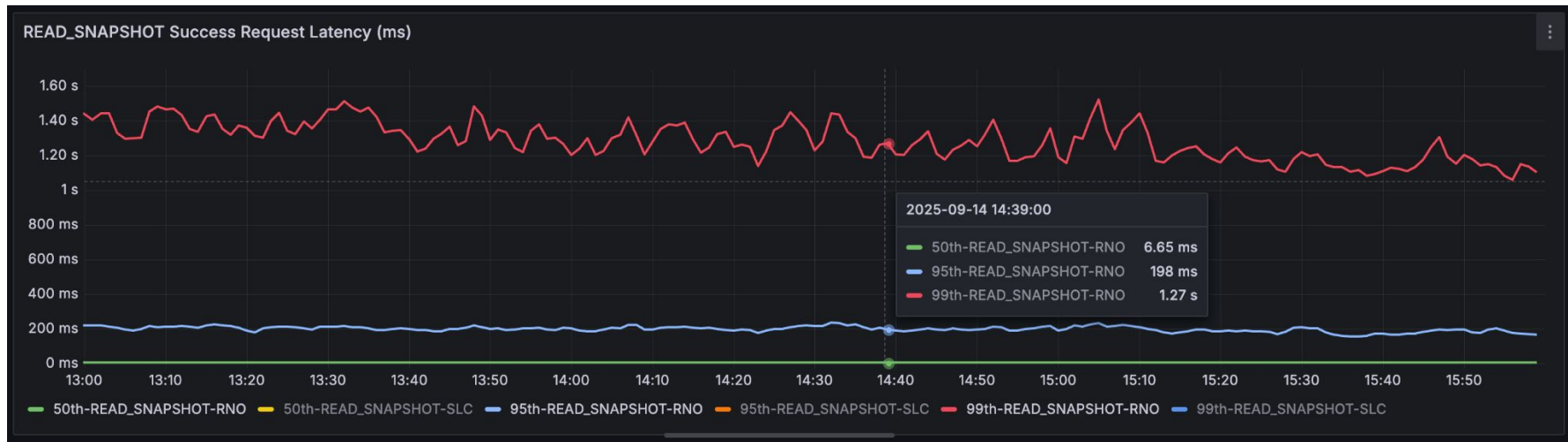
- Query result (AU airports) is stored in one cache entry
 - Key: <QT>:1:IsActive=true&Country=AU
- 1 getRange + 1 cache lookup

Find all AU airports with active flight routes from LAX

```
g.V().has('code', 'LAX').outE().has('IsActive', 'true').inV().has('country', 'AU')
```



Without cache: High 95th and 99th latency



One-Hop Sub-Query Result Cache

- Divide the query into subqueries.
- Select the subquery(ies) that dictates the original query latency to cache.
- Processing the original query includes processing parts of it using graph processing, and parts of it using cache.
- Example:

Find all countries with direct flights to AU airports that LAX also flies to directly.

```
g.V().has('code', 'LAX')  
    .outE('route').has('isActive', 'true').inV().has('country', 'AU') // first-hop cache  
    .inE('route').outV().values('country').dedup() // second-hop cache
```

- Cache Template 1: `outE('route').has('isActive', ?).inV().has('country', ?)`
- Cache Template 2: `inE('route').outV().values('country')`
- ***Why dedup() is not in Cache Template 2?***

`g.V().has('code', 'LAX')`

`.outE('route').has('isActive', 'true').inV().has('country', 'AU')`

`.inE('route').outV().values('country').dedup()`

Without cache:

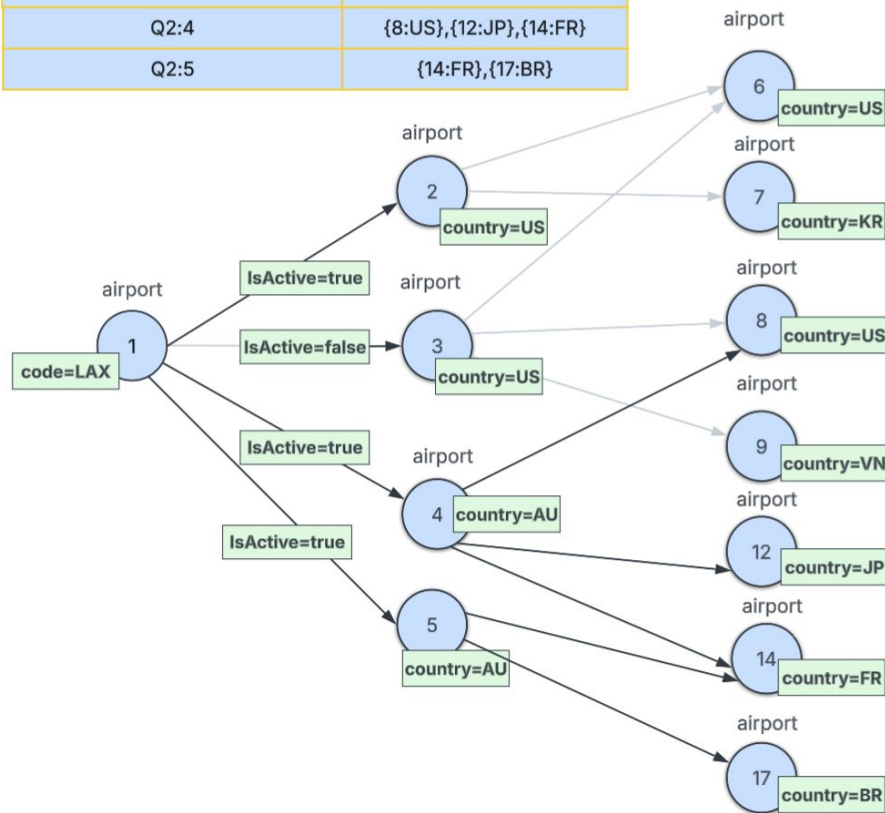
- 1 `getRange` for index lookup
- 1 `getRange` for fetching LAX edges
- 4 `getRange` for fetching 'country'
- 2 `getRange` for fetching edges
- 4 `getRange` for fetching 'country'

With cache:

- 1 `getRange` for index lookup
- 1 cache lookup:
`Q1:1:isActive=true&country=AU`
- 2 cache lookups for `Q2:4` and `Q2:5`

Cache Entries

Q1:1:isActive=true&country=AU	4,5
Q2:4	{8:US},{12:JP},{14:FR}
Q2:5	{14:FR},{17:BR}



1 Big Database getRange() vs 1 Cache Get()

Mode	Average (ms)	95th (ms)	99th (ms)	Latency Reduction
getRange	9.7	11.4	12.5	
FDB cache get	2.2	2.2	2.2	~80%
memcached cache get	1.2	1.24	1.25	~87%

Table 1. Benchmarking 1 big getRange() vs 1 cache get() (10k values in size)

Multiple Small Database getRange() vs Multiple cache get() Issued in Parallel

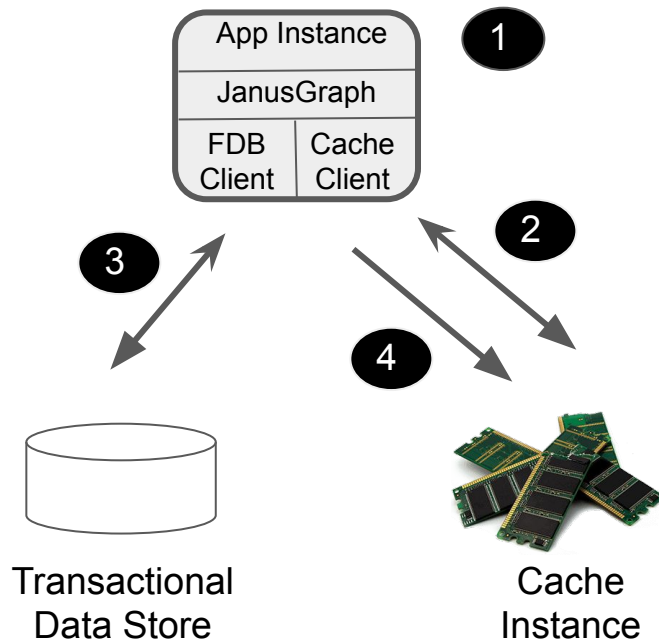
Mode	Average (ms)	95th (ms)	99th (ms)	Latency Reduction
getRange	221	231	253	
FDB cache get	141	143	158	~36%
memcached cache get	54	54	54	~76%

Table 2. Benchmarking 10k small getRange() vs 10k cache get()

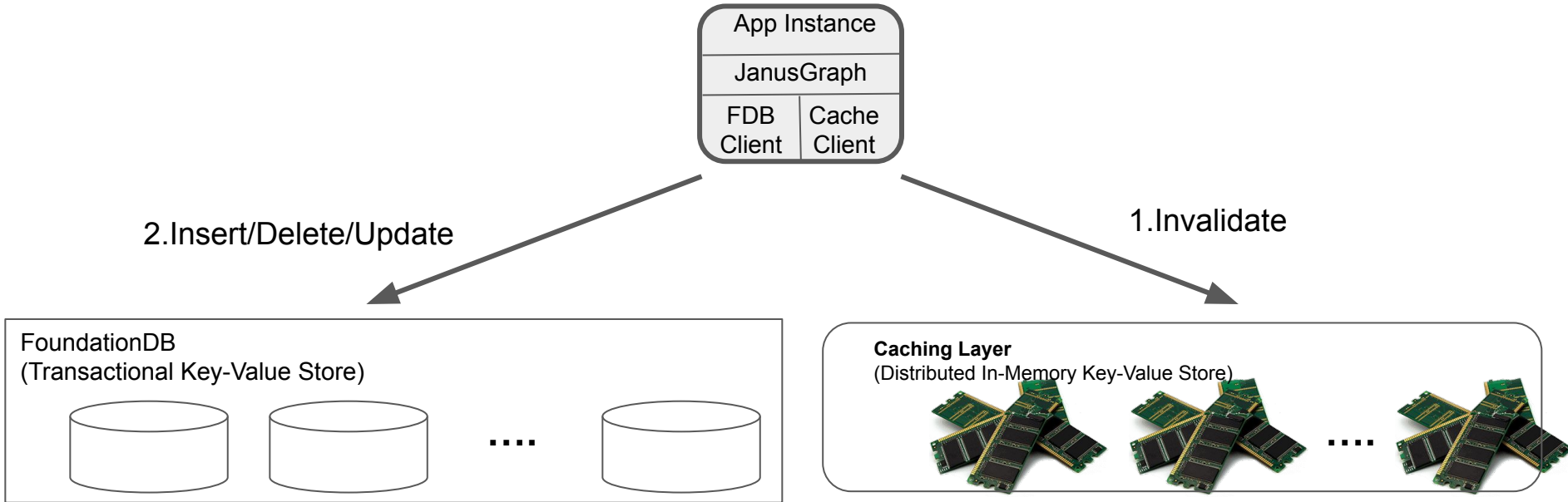
Read Request with Cache

Given a graph traversal query:

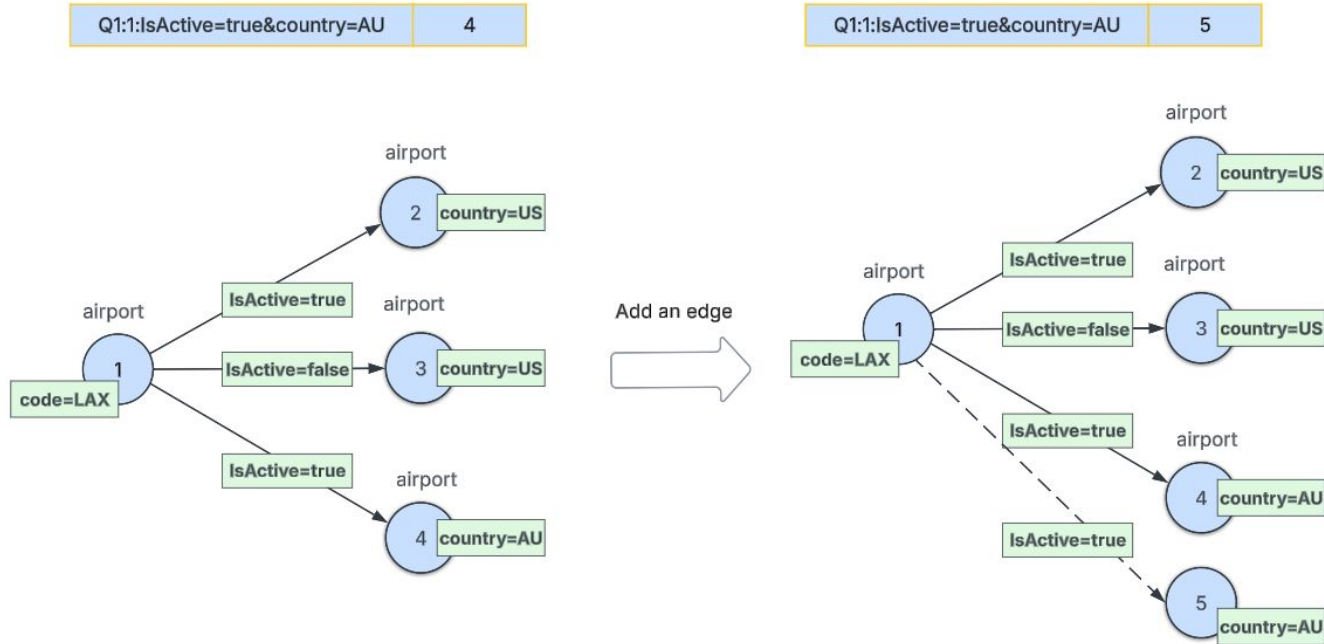
1. Decide to cache/not cache based on a predefined set of query templates
2. Compute the corresponding cache key. Look-up the cache. If the value is there, return the value as the query result
3. If the value is missing, process the query via JanusGraph
4. Store the query result in the cache (along with the key identified at Step 2) and return the query result



Write Request with Cache (write-around)



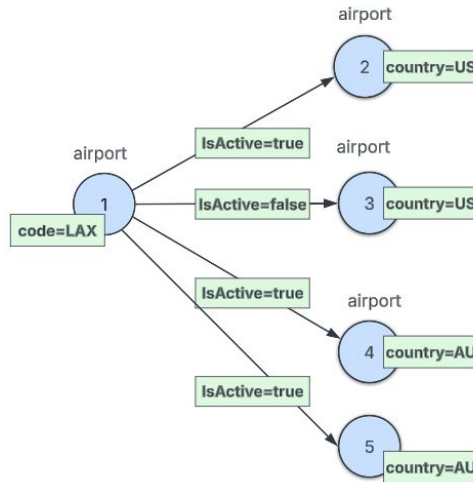
Case 1: Add/Remove an edge



- Impact at most 2 cache keys (per cache template)

Case 2: Update an edge

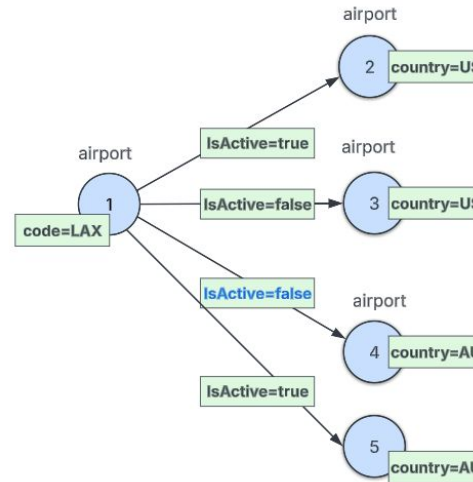
Q1:1:IsActive=true&country=AU	4,5
Q1:1:IsActive=false&country=AU	



Update an edge



Q1:1:IsActive=true&country=AU	5
Q1:1:IsActive=false&country=AU	4

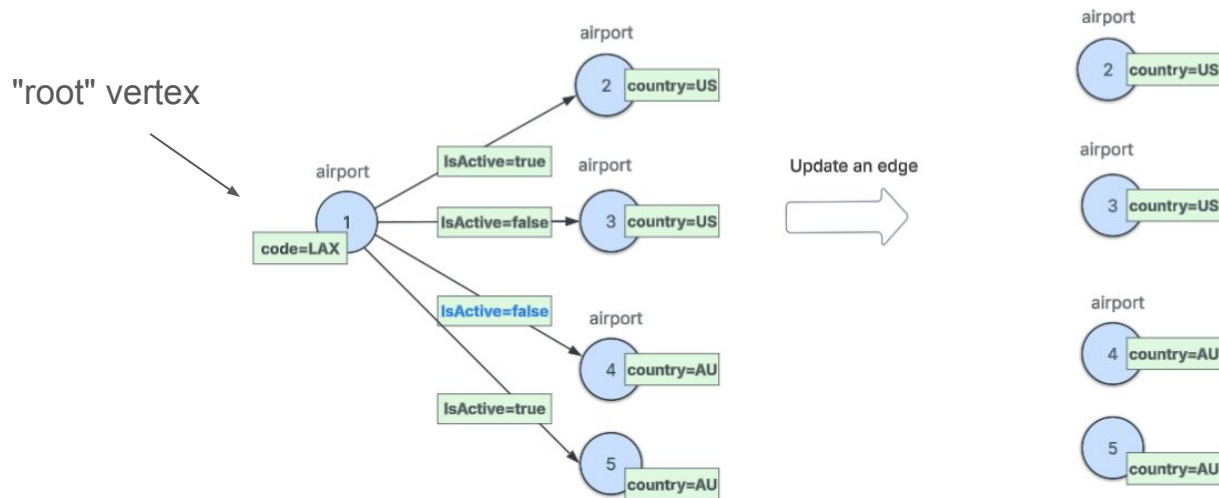


- Impact at most 4 cache keys (per cache template)

Case 3: Update/Remove a root vertex

clearRange(Q1:1)

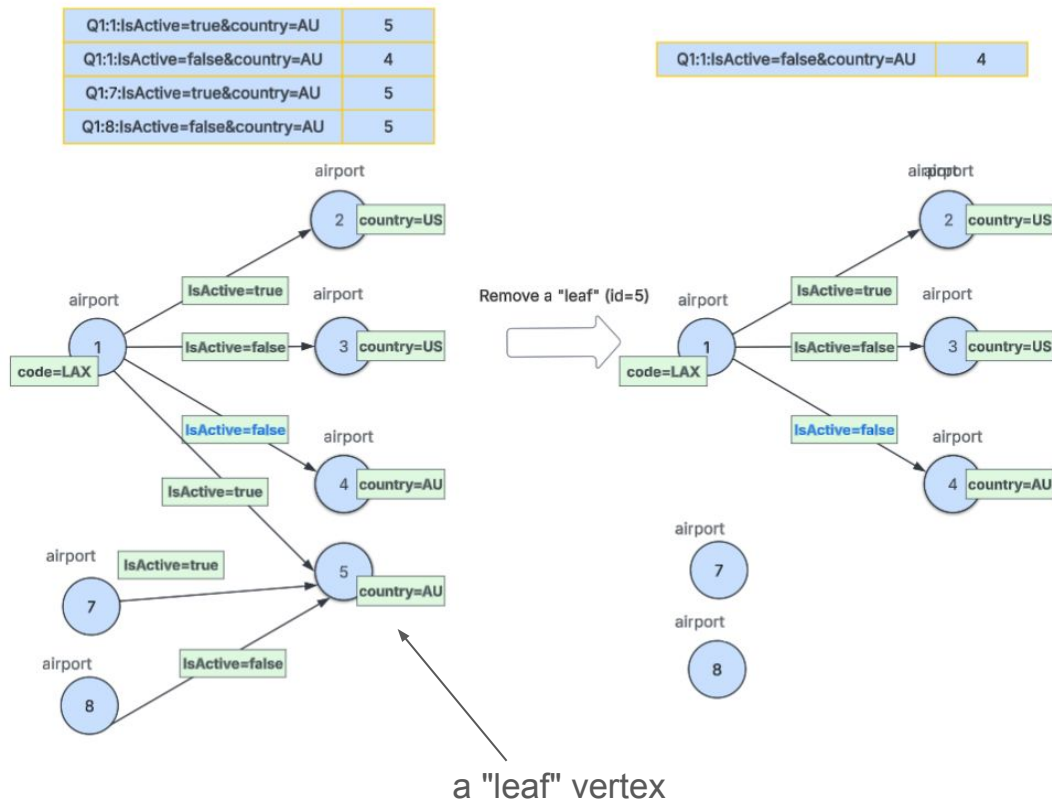
Q1:1:IsActive=true&country=AU	5
Q1:1:IsActive=false&country=AU	4



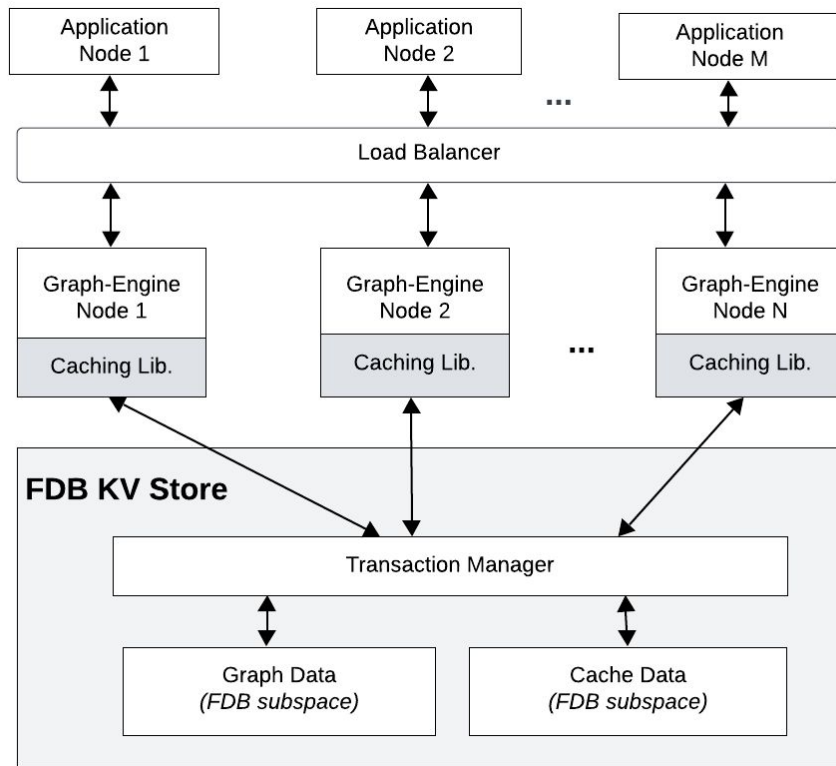
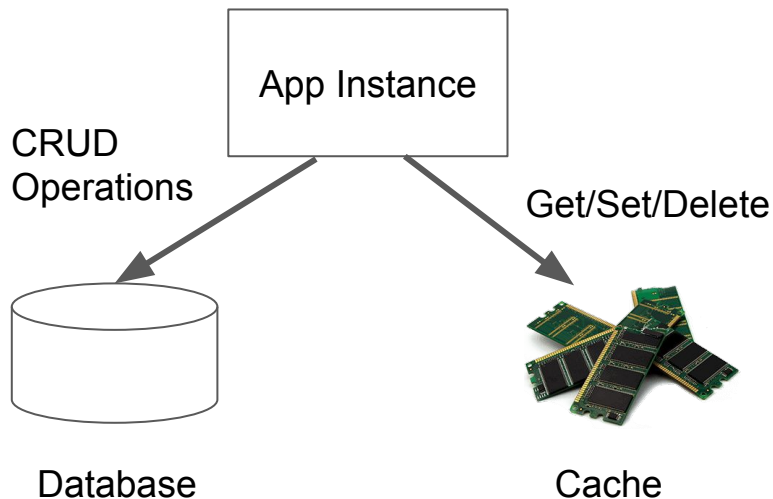
- Impact at most N cache keys (per cache template)
- Quick to invalidate based on prefix

Case 4: Update/Remove a leaf vertex

- More expensive (up to 2N impacted cache keys)
- Exists today when an application deletes a supernode vertex
- Extra-constraints to prevent this case to be expensive:
 - If a vertex needs to be removed, its edges must be removed in small batches first
 - If a super-node vertex property needs to be updated?



Implementation: FoundationDB as Cache Storage



One-Hop Sub-Query Result Cache - Summary

- Reduce read-only queries significantly
- Reasonable overhead on write queries
- Each transaction includes both operations for graph data and cache data, ensuring strong consistency in read-write workloads

Full-Query Result Cache

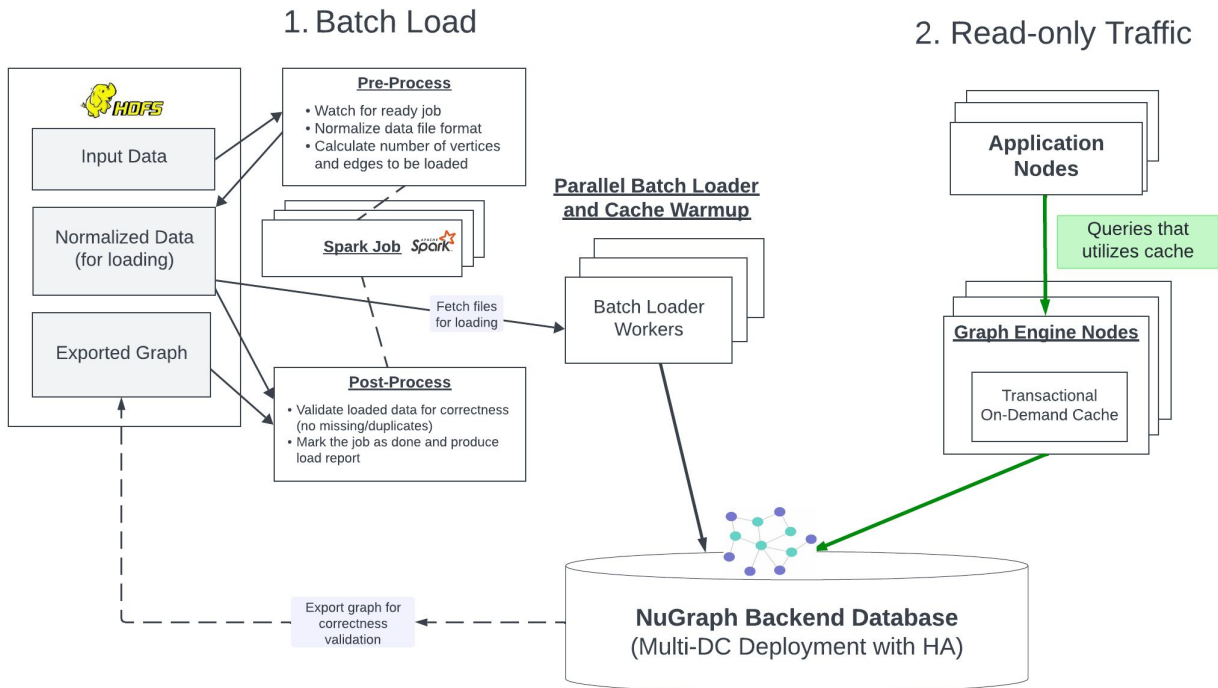
Query Template: `g.V.has('name', ?).has('age', between(3, 10))
 .outE('knows').has('year', P.gt(?))
 .inV().where(outE('locates_at').inV().has('location', ?))
 .limit(100).valueMap(true)`

Query: `g.V.has('name', 'john doe').has('age', between(3, 10))
 .outE('knows').has('year', P.gt(20))
 .inV().where(outE('locates_at').inV().has('location', 'US'))
 .limit(100).valueMap(true)`

Cache Key: `FQ1:<john doe>:20:<US>`

- Pros: Straightforward to implement (read path) and reduce latency significantly
- Cons: Cache invalidation is very expensive
- Used for batchload-then-read-only-traffic workloads at eBay

Example: Batchload-then-Read-Only-Traffic Pipeline



Machine learning applications

Summary

- FoundationDB is a distributed key-value store that is horizontally scalable and strong consistency
- FDB uses MVCC and OCC to manage transactions
- Its design follows a "divide and conquer" approach where each module is responsible for a specific task
 - Each module can be scale independently with other modules
 - Each module can have special hardware tailored for the module's needs
 - Easier to detect bugs and debug
- eBay has developed graph database service using FDB as the storage backend and successfully managed graphs with billions of vertices and edges
- Cache solutions are practical for graph and significantly reduce query latency

References

1. FoundationDB paper: <https://www.foundationdb.org/files/fdb-paper.pdf>
2. JanusGraph doc: <https://docs.janusgraph.org/>
3. Tinkerpop doc: <https://tinkerpop.apache.org/docs/current/reference/>
4. Gremlin query language: <https://arxiv.org/abs/1508.03843>
5. Practical Gremlin book:
<https://www.kelvinlawrence.net/book/PracticalGremlin.html>

Q & A



Consistency Models

- Strict serializability
- Serializability
- Linearizability
- Read snapshot
- Eventual consistency
- Causal consistency
- Read-your-write consistency

eBay - USC Research Collaboration

- Funded by eBay eRUPT program
- Our progress in the last two years is in the topic of graph cache
 - 2023
 - Full-query-result cache
 - 2024
 - Sub-query-result cache (paper to be submitted)
 - New architecture for strong consistent and scalable cache
 - 2025
 - Dataflow graph query processing engine engine (in progress)
- We are looking for the best students!
 - Strong background and experience in databases and distributed systems
 - Require strong commitment to the project for it to be success
 - Chances for internship/full-time job offer at eBay

How to Apply?

1. Email me if you are interested
2. A task will be assigned. After completing the task, email me at hieunguyen@ebay.com with:
 - The delivery materials required by the task
 - Your CV

Feel free to contact me if you have any questions on the tasks.

Graph Databases

- Amazon Neptune
- DGraph
- **JanusGraph**
- NebulaGraph
- Neo4J
- TigerGraph

JanusGraph Quizz

What is the storage overhead of adding one vertex-centric index for an edge property?

Does composite index support range query?

Does vertex-centric index support range query?

Does mixed index support range query?